

Caterpillar Asset Management V2 API — Briefing

Prepared as a discussion guide. Based on the OpenAPI v2.1.0 spec, the Asset Management V2 Developer Guide (SV-0066), and live validation against the production endpoint.

1. Executive summary

The Asset Management V2 API lets a dealer **read and manage who owns a piece of equipment** in Caterpillar's systems. It is a small, focused API — **four operations** — but it sits on top of a rich ownership model (customers, dealers, DCNs, ownership types, and transfer workflows).

In one sentence: **it is the system of record for “which dealer/customer owns which asset,” and the API both answers that question and lets dealers change the answer.**

Three things make it strategically interesting:

1. **It's authoritative.** Search returns real, live ownership data across the entire dealer network — not a dealer-local copy.
2. **It encodes business rules, not just data.** Adding an “owned” record can automatically trigger a cross-dealer transfer workflow. The API enforces Caterpillar's ownership-conflict policy for you.
3. **It's integration-ready.** OAuth2 + JSON over HTTPS means it drops into Copilot, Excel, Snowflake, Power BI, or any internal app with minimal glue.

2. The domain: what the data actually represents

A few concepts are worth defining up front, because every endpoint speaks in these terms.

Term	Meaning
Asset	A physical machine. Uniquely identified by makeCode + serialNumber .
Make code	Manufacturer code (e.g. <code>CW1</code>). Dealers can also use a 2-char dealerMakeCode .
Serial number	The asset's serial. CAT serials have a strict format (3 alphanumeric + 5 numeric); non-CAT serials are looser.

Term	Meaning
CCID	Caterpillar Customer ID — the end customer.
DCN	Dealer Customer Number — a specific dealer↔customer relationship.
Dealer code	The dealer organization (e.g. <code>B150</code> = Cleveland Brothers). Passed as partyNumber .
Ownership record	One association between an asset and a customer, uniquely keyed by makeCode + serialNumber + DCN + dealerCode .
Ownership type	<code>owned</code> , <code>rental</code> , <code>leased</code> , <code>sold</code> , <code>inventory</code> , <code>unknown</code> .
Relation status	<code>ACTIVE</code> (current) or <code>PENDING</code> (a transfer is in progress).

Key insight for the meeting: one asset can have **many** ownership records at once — different dealers, different DCNs, different ownership types. The rule that makes it coherent: **only one customer can have an ACTIVE + OWNED record at a time**. Everything else (leased, rental, inventory) can coexist. The conflict logic in §5 exists to enforce that single rule.

3. Authentication & environments

- **Auth:** OAuth2 **client-credentials** via Microsoft Entra ID. The dealer's app gets a client ID + secret, requests a bearer token, and includes it on every call. **Tokens last 60 minutes** and must be refreshed.
- **Per-dealer credentials & entitlements.** Credentials are issued per dealer code and are *entitlement-enforced* — you can only act on data you're entitled to. Trying to act as a dealer code you aren't provisioned for returns `403.113` (we confirmed this in testing).
- **No sandbox.** This is the single most important operational fact: **the v2 endpoint is production. There is no separate test environment.** Caterpillar instead issues an *isolated test dealer code* plus test assets/DCNs that live inside production. You validate against real infrastructure, fenced to a test dealer.
- **Response format:** JSON. No pagination — searches return all matches at once.

4. The four endpoints

All four are `POST`. The asset identifier (partyNumber + serialNumber + make code, plus DCN where relevant) travels as **query parameters**; the payload travels in the JSON body.

4.1 Search ownership records — `POST /ownershipRecords/search`

What it does: returns the ACTIVE/PENDING ownership records matching a filter.

How it works: you supply 1–2 exact-match filters on one of four fields — `dcn`, `serialNumber`, `assetName`, `makeCode`. Two filters combine with logical **AND**. (`makeCode` can't be used alone — it needs a second filter.) You can sort by up to three fields and trim the response to just the attributes you need.

What you get back: for each record — the asset metadata (make, model, model year, product family), the customer (CCID + name), and the dealer association (dealer, DCN name, ownership type, status, subscription flag).

The big caveat: search is **network-wide**. A serial-number search returns records across *all* dealers, not just yours. We saw a single serial return six records spanning five dealers. This is powerful (full visibility) but means you must filter/interpret results carefully.

4.2 Add or update ownership record — `POST /ownershipRecords`

What it does: creates a new ownership record, updates an existing one, or re-activates a previously expired one.

How it works:

- If the record **exists**, only the fields you send are updated.
- If it **doesn't exist**, three fields become required: `ownershipTypeCode`, `model`, `modelYear`.
- Asset metadata (model, year, product family) can only be changed by the **current owning dealer**, and Caterpillar may override it from its own systems.

Why it's more than a CRUD insert: setting ownership to `owned` triggers conflict detection (\$5). So “add a record” can mean “claim ownership,” which can ripple into a transfer workflow.

4.3 Expire ownership record — `POST /ownershipRecords/expire`

What it does: ends an ACTIVE or PENDING ownership record (the “remove ownership” operation). Expiring a PENDING record also **cancels** its associated transfer request.

How it works: identify the asset (serial + make + DCN) and call it. Returns `204 No Content` on success. This is how a dealer relinquishes an association — e.g. when equipment is sold or the customer relationship ends.

4.4 Approve / reject transfer request — `POST /ownershipRequests/transfer`

What it does: resolves a pending cross-dealer ownership transfer.

How it works: only the dealer that **currently owns** the asset can act.

- **APPROVED** → the current owner's record is expired and the requesting dealer becomes the new owner.
- **REJECTED** → ownership stays put; the pending record is removed. **A reason is required to reject.**

This is the human approval step in the cross-dealer workflow that §5 sets up.

4.5 Adding a record in practice — minimum fields and make codes

Because “add a record” is the operation we'll most often automate, it's worth spelling out exactly what it requires.

Minimum to create a *new* record (the three body fields are required *only* because the record is new — updates send just the fields that change):

- **Identity (query parameters):** `partyNumber` (dealer code), `serialNumber`, `makeCode` or `dealerMakeCode`, and `dcn`.
- **Body (required for a new record):** `ownershipTypeCode`, `model`, `modelYear`.
- **Body (optional):** `productFamilyCode`, `productFamilyName`, `baseAssetName`, `customAssetName`.

Preconditions that must already be true, or the call is rejected:

- The **DCN already exists, is active, and is tied to a valid customer (CCID)**. This is the most common stumbling point: *the API attaches an asset to an existing dealer–customer relationship; it does not create the customer or the DCN*. Those are established in the Customer Admin Tool first.
- The **make code is a recognized manufacturer**, the **serial matches the required format**, and your **credentials are entitled** to that dealer code.

Make codes — a field we have to get right. Every asset is keyed by *make code

- `serial number*`, so a make code is mandatory on every add. Two forms exist; provide exactly one:

Field	Format	What it is	Examples
<code>makeCode</code>	3 chars	Caterpillar's master manufacturer code	<code>CW1</code> (CAT), <code>KDC</code> (Komatsu), <code>FA1</code> (Ford), <code>G02</code> (Galion), <code>SB6</code> (Snow Wolf)
<code>dealerMakeCode</code>	2 chars	A dealer's own shorthand for a make	<code>CW</code> , <code>AA</code>

Four implications worth planning around:

1. **It must be valid/recognized** — you can't invent a code (`400.202` / `400.209` for `makeCode` , `400.208` for `dealerMakeCode`).
2. **CAT and non-CAT (competitive) makes are both tracked**, each with its own code — a Komatsu asset needs the Komatsu code, not a generic CAT one.
3. **The make drives serial-number validation** — CAT makes enforce the strict 8-character serial format (3 alphanumeric + 5 numeric); non-CAT makes allow the looser 4–50 character format. The wrong make can cause a valid serial to be rejected.
4. **Dependency to line up:** this API *validates* make codes against Caterpillar's reference data but does not *return a list of them*. To populate the field reliably at scale we need a source of truth for valid make codes and our dealer make-code mappings. That lookup is a prerequisite for any bulk add / onboarding integration.

5. The clever part: automatic conflict resolution

This is the concept most worth being able to explain in the room, because it's where the API stops being a database and starts being a *workflow engine*.

When a dealer adds an **owned** record for an asset, the API checks for conflicts:

- **No conflict** → the record becomes `ACTIVE` immediately.
- **Conflict within your own dealership** (another DCN under you owns it) → the conflicting record is **automatically expired**, and your new record goes `ACTIVE` . Silent, immediate.
- **Conflict with a different dealer** (another dealer currently owns it) → the API **does not** seize ownership. Instead it creates a **PENDING ownership transfer request**, and your record sits in `PENDING` until the current owner **approves** it via the transfer endpoint.

So the four endpoints aren't four isolated CRUD operations — they're the steps of a **negotiated transfer protocol**:

```
search (find current owner)
  → add "owned" (request it)
    → [same dealer]      auto-expire + ACTIVE      (done)
    → [different dealer] PENDING transfer request created
                        → owner: approve → you become owner
                        → owner: reject  → ownership unchanged
```

Talking point: *"You can't accidentally steal an asset from another dealer — the API turns that into a request the other dealer has to approve."*

6. Capabilities and limits (set expectations honestly)

What it does well:

- Authoritative, network-wide ownership lookup.
- Safe, policy-enforced ownership changes and transfers.
- Clean machine-to-machine auth that fits modern tooling.
- Fine-grained response shaping (request only the fields you need).

What it deliberately does *not* do (good to pre-empt in the meeting):

- **Exact-match search only.** No partial, wildcard, or fuzzy matching. You search by an exact DCN/serial/name/make — not "all 980H loaders."
- **Only four filterable fields.** You cannot filter by model, model year, customer, ownership type, or status — only `dcn`, `serialNumber`, `assetName`, `makeCode`. (We verified the API rejects anything else.)
- **Max two filters, AND only.** No OR logic, no >2 conditions.
- **No pagination / no bulk list.** You must search by a known value; there's no "give me everything" call.
- **DCN numbers are withheld for other dealers' records.** You see the customer *name* (`dcnName`) but not the DCN *identifier* for records that aren't yours — a privacy boundary.

These aren't bugs; they're a tightly-scoped, security-conscious design. The framing for the meeting: *"It's a precision instrument, not a reporting warehouse."*

7. Use cases & integration opportunities

The API is small, but because it's authoritative and well-secured, it unlocks a lot. Grouped by audience:

Operational / dealer staff

- **Ownership lookup tool** — paste a serial, see who owns it across the network.
- **Onboarding/offboarding automation** — when a sale or return happens in another system, automatically add or expire the ownership record.
- **Transfer inbox** — surface incoming transfer requests for a dealer to approve or reject, instead of checking the Admin Tool manually.

Conversational / AI

- **M365 Copilot agent** (built) — “Who owns serial 2WS23456?” answered in chat; guided add/expire/transfer with confirmations. Demonstrates natural-language access to the ownership system.

Desktop / analyst

- **Excel integration** (built) — a `=CatLookupSerial()` worksheet function and a batch macro that takes a column of serials and returns a full results sheet. Lets non-developers use the API with zero code.

Data platform

- **Snowflake UDF** (designed) — wrap search as a SQL function so ownership data can be joined against other datasets directly in the warehouse, with the OAuth token handled natively by Snowflake.
- **Power BI / reporting** — enrich existing equipment reports with live ownership and subscription status.

Data quality / governance

- **Reconciliation jobs** — compare the dealer’s internal records against the authoritative ownership data and flag drift.
- **Subscription insight** — the `dcnHasSubscription` flag exposes whether an ownership record has an active subscription, useful for renewals/coverage gaps.

The pattern across all of these: **one small API, many surfaces**. The same four endpoints power chat, spreadsheets, SQL, and dashboards.

8. Risks, gotchas, and questions to raise

Good items to bring up proactively — they signal you understand the operational reality, not just the happy path.

- **Production-only, no sandbox.** Any write touches production. Mitigation: Caterpillar’s isolated test dealer code + test assets. Action item: confirm our test data and a safe validation plan before any write integration ships.
- **Writes have side effects beyond the call.** Adding an “owned” record can generate a real transfer request to another dealer. Integrations must treat writes as deliberate, confirmed actions — never automatic loops over search results.
- **Credential & secret management.** Client secret + 60-min tokens. Where does the secret live (server-side broker vs. embedded), and who rotates it? Embedding it in distributed tools (e.g. a shared spreadsheet) is a leak risk.
- **Entitlement provisioning timeline.** Credentials start scoped to a test dealer code and are upgraded to real data later (a ~1–2 week process). Plan rollout around that.

- **Search ergonomics.** Because search is exact-match on four fields, end-user tools need to steer people to supported queries (serial/DCN), or pre-resolve fuzzy input before calling.
- **Rate limits.** The API returns 429 when throttled; bulk tools need retry/ backoff. Open question: what are the documented limits for our tier?

9. Ready-to-say talking points

- *"It's four endpoints — search, add/update, expire, and transfer — but it's the authoritative source for asset ownership across the whole dealer network."*
- *"Search is read-only and network-wide; the other three are writes that the API governs with built-in conflict rules."*
- *"The smart part is conflict handling: claiming an asset another dealer owns doesn't take it — it opens a transfer request they have to approve."*
- *"It's exact-match search on four fields by design — a precision lookup, not a reporting tool. We'd pair it with our own data for analytics."*
- *"Auth is standard OAuth2, so it plugs into Copilot, Excel, Snowflake, and Power BI with little effort — we've already prototyped the first three."*
- *"There's no sandbox; everything runs in production against an isolated test dealer code, so our rollout plan has to respect that."*

Appendix A — Endpoint quick reference

Operation	Method & path	Body	Success	Notes
Search	POST /ownershipRecords/search	filters + response attrs	200 + records	read-only, network-wide
Add / update	POST /ownershipRecords	asset metadata	200 (ACTIVE/PENDING)	new records need type+model+year
Expire	POST /ownershipRecords/expire	none	204	cancels a pending transfer too

Operation	Method & path	Body	Success	Notes
Transfer	POST /ownershipRequests/transfer	status + reason	204	owner-only; reason required to reject

Appendix B — Common error codes

Code	Meaning
400.202	Invalid/missing manufacturer (make) code
400.204	Invalid/missing DCN
400.210	DCN not found or not tied to a valid CCID
400.216	Invalid model year
400.217	Asset ownership conflict
400.218	A pending transfer already exists
403.113	Credentials not entitled to this operation/dealer
429.x	Rate limit exceeded

Appendix C — Glossary

- **CCID** — Caterpillar Customer ID (the end customer).
- **DCN** — Dealer Customer Number (a dealer↔customer relationship).
- **partyNumber** — the dealer code the call acts as.
- **ACTIVE / PENDING** — current ownership vs. a transfer in progress.
- **Customer Admin Tool** — the web app where all of these actions can be viewed and validated.